

AI in Production — What Actually Breaks



A notebook that worked once isn't done yet

Gourav Shah · School of DevOps & AI · Lesson + Lab + Quiz



Why doesn't "it works in my notebook" mean it's done?

A demo you gave once is not the same demo run for strangers

a notebook run

once, for yourself

data whose shape you
already knew

production

the same demo,
thousands of times a day
for people you'll never meet

someone has to notice within minutes when it stops working

Everything in this module is about that gap



The notebook-to-production gap

Cook it once for yourself, or put it on a restaurant menu



You taste as you go; a stranger can't

A trained model is the recipe — serving it adds four things

- 1 someone else's environment runs your code
- 2 input arrives that you never previewed
- 3 failures get caught and logged, not just shown in a cell
- 4 every request gets handled, not just the ones shaped like your test data

Today's lab makes the first one concrete

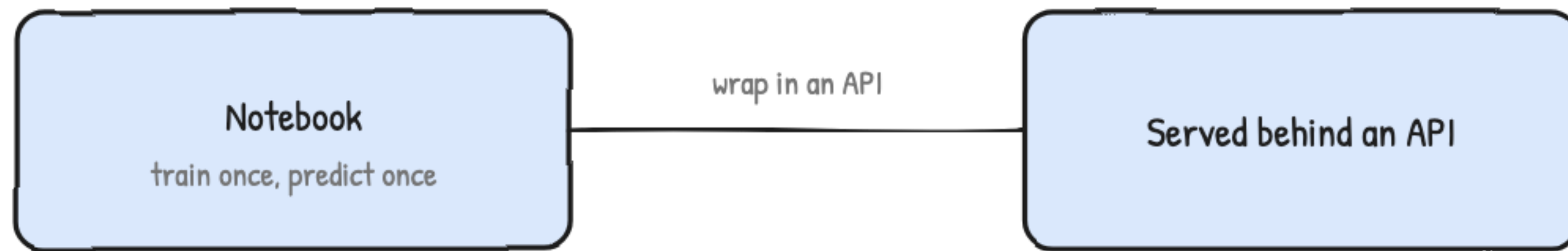
One loop a notebook never needed, and production can't run without

Notebook

train once, predict once

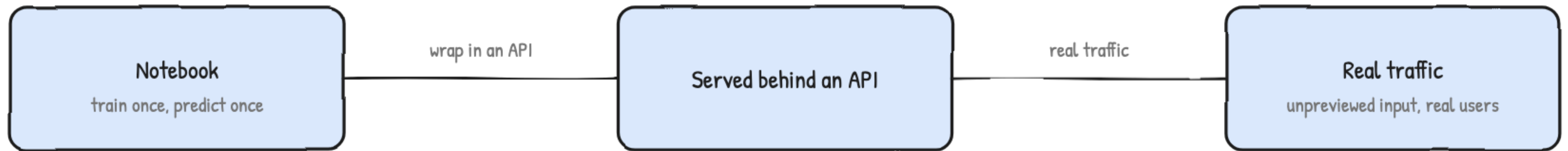
That loop is what a notebook never needed

One loop a notebook never needed, and production can't run without



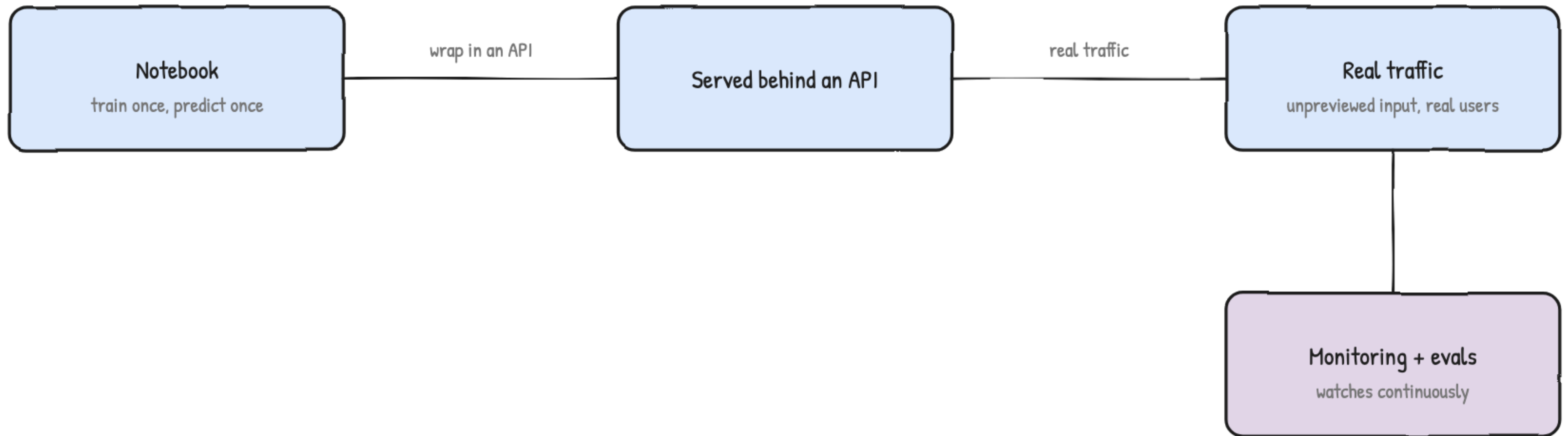
That loop is what a notebook never needed

One loop a notebook never needed, and production can't run without



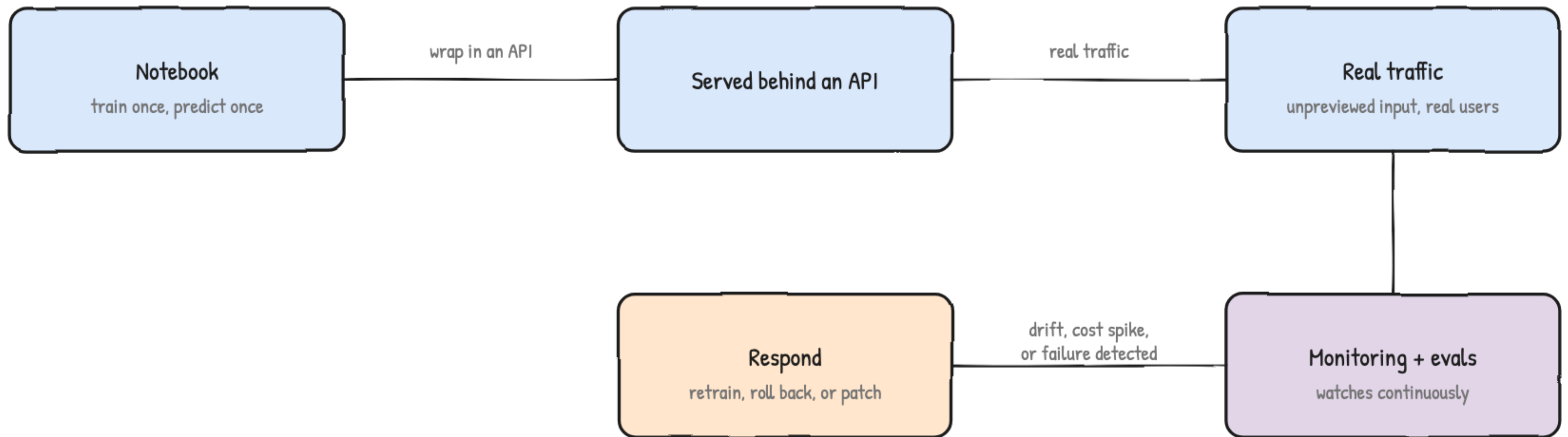
That loop is what a notebook never needed

One loop a notebook never needed, and production can't run without



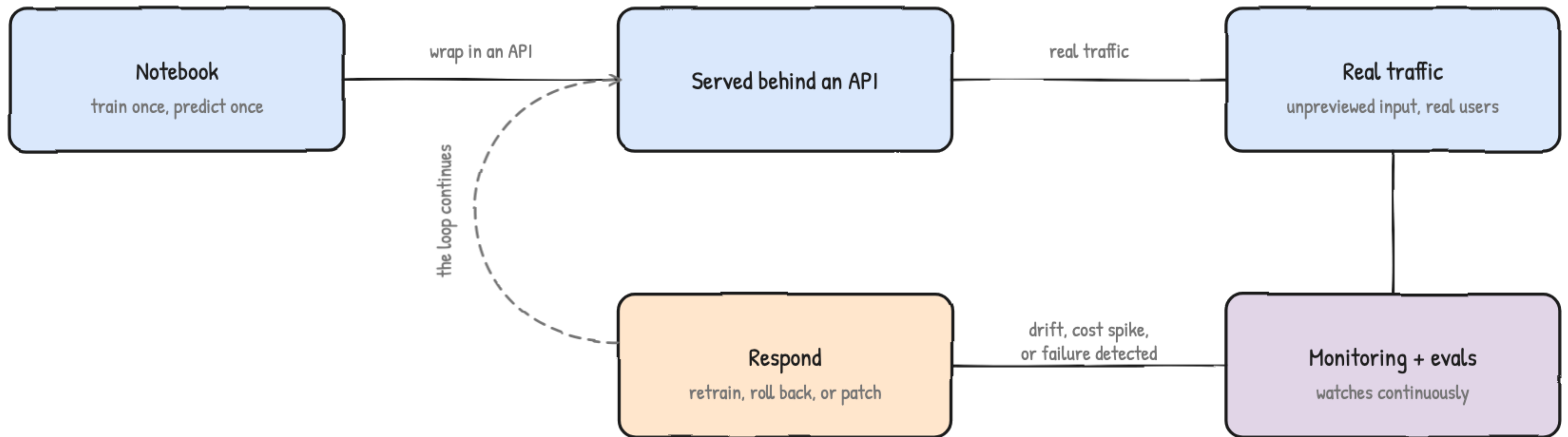
That loop is what a notebook never needed

One loop a notebook never needed, and production can't run without

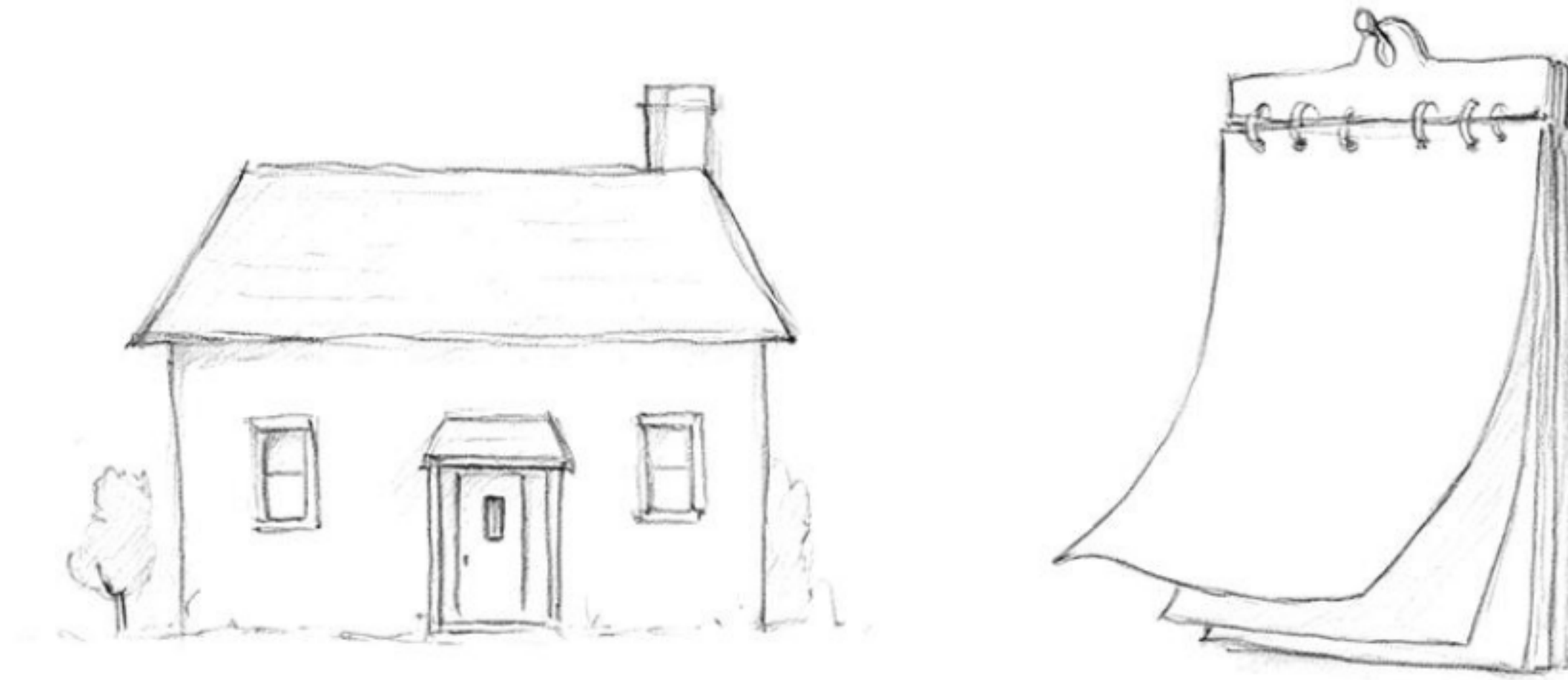


That loop is what a notebook never needed

One loop a notebook never needed, and production can't run without

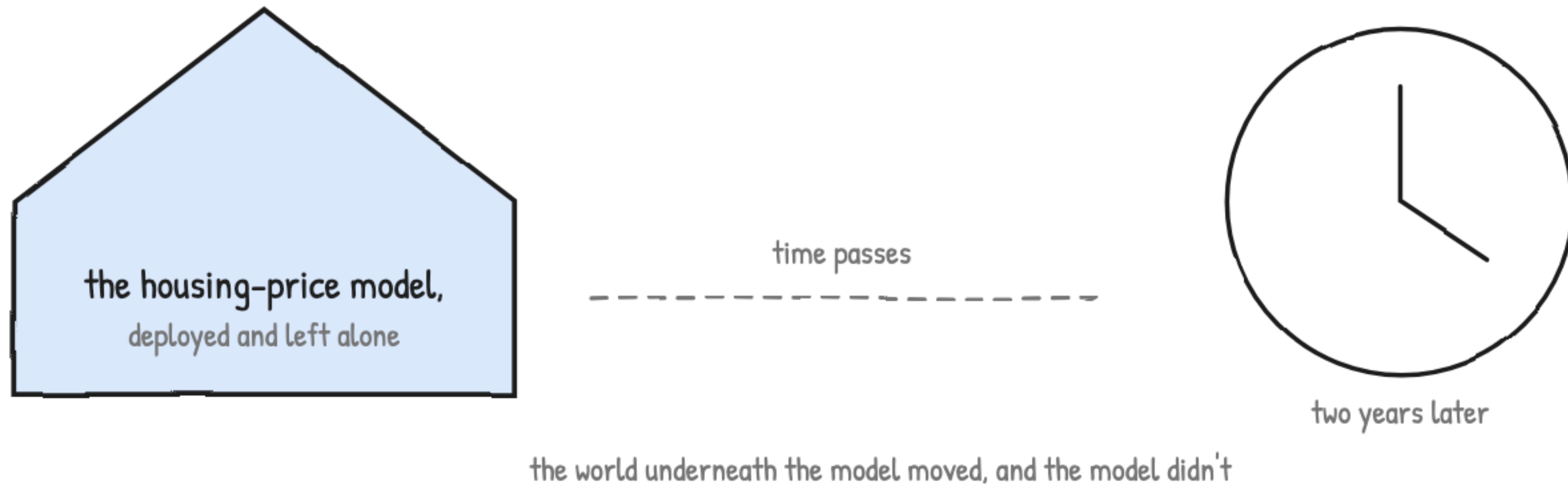


That loop is what a notebook never needed



Drift and degradation: the ground moves under a working model

Nobody touched the code — the market underneath it moved



Nobody touched the code; the market moved

Drift is the cause; a close cousin is data drift

drift

the real relationship between
inputs and the right answer
changes after you deployed

data drift

the inputs themselves start
looking different from what
the model trained on

Same root cause, two different symptoms

Degradation is what you actually observe

the model —
byte-for-byte the one
you shipped, nothing changed

real-world accuracy, sliding, over time

A line graph with a red line sloping downwards from left to right, representing a decrease in accuracy over time.

the train/test split only proves accuracy at one point in time

The model didn't change; the accuracy did



Hallucination and cost, at scale

Caught once is an annoyance — unattended at scale is different

one wrong-but-confident sentence

caught by you,
reading a single reply

a minor annoyance

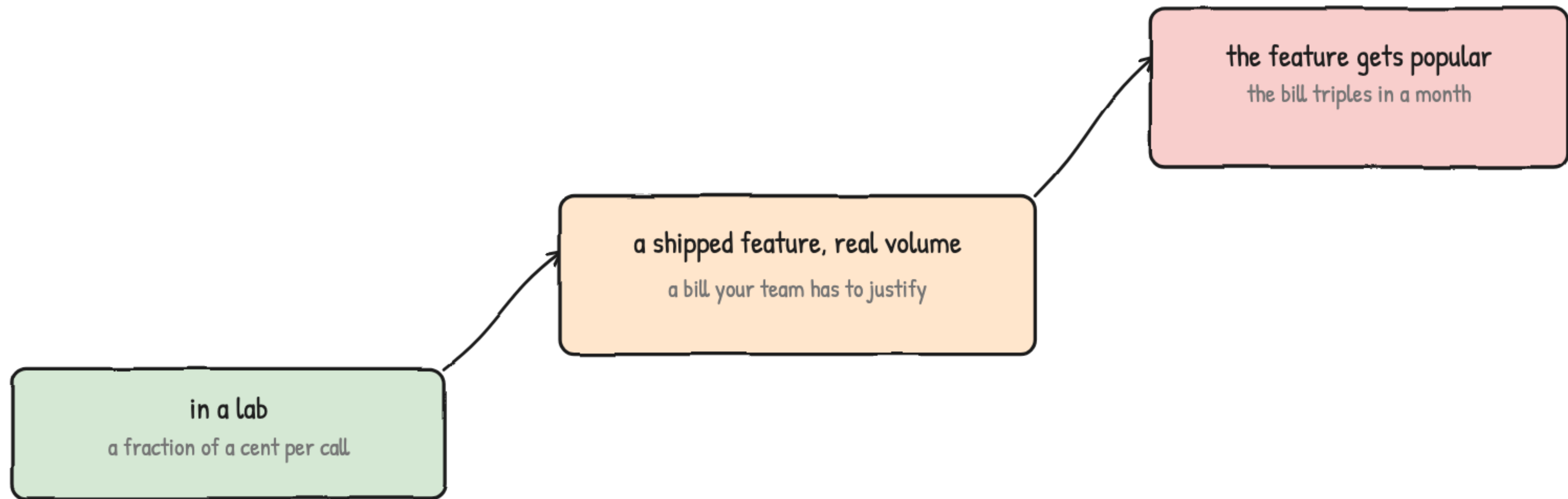
the same mechanism, unattended

ten thousand customer
questions a day

reaches a customer first

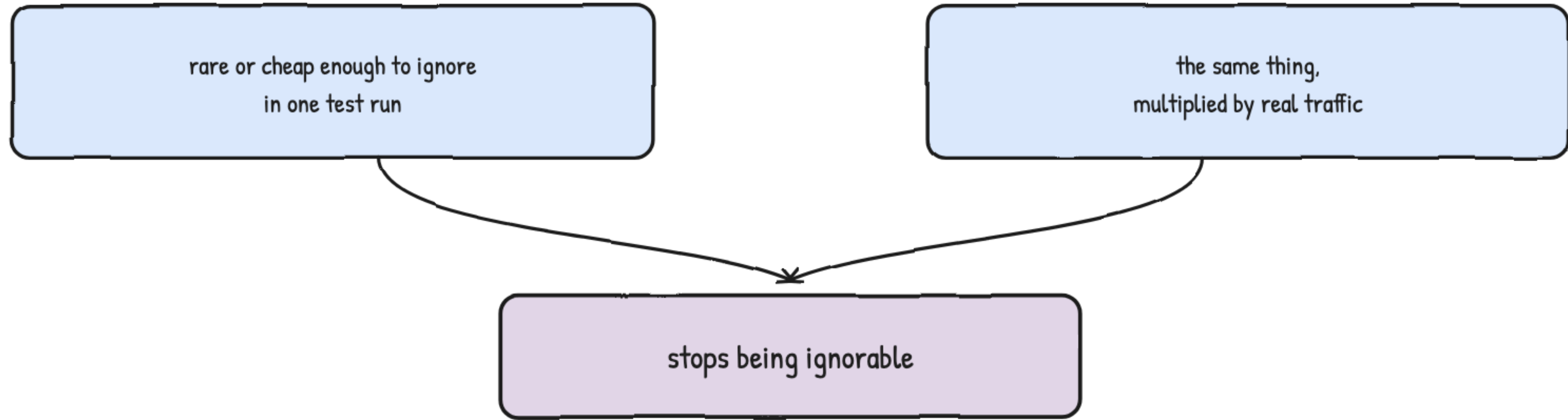
It reaches a customer before anyone checks

A fraction of a cent becomes a bill your team has to justify



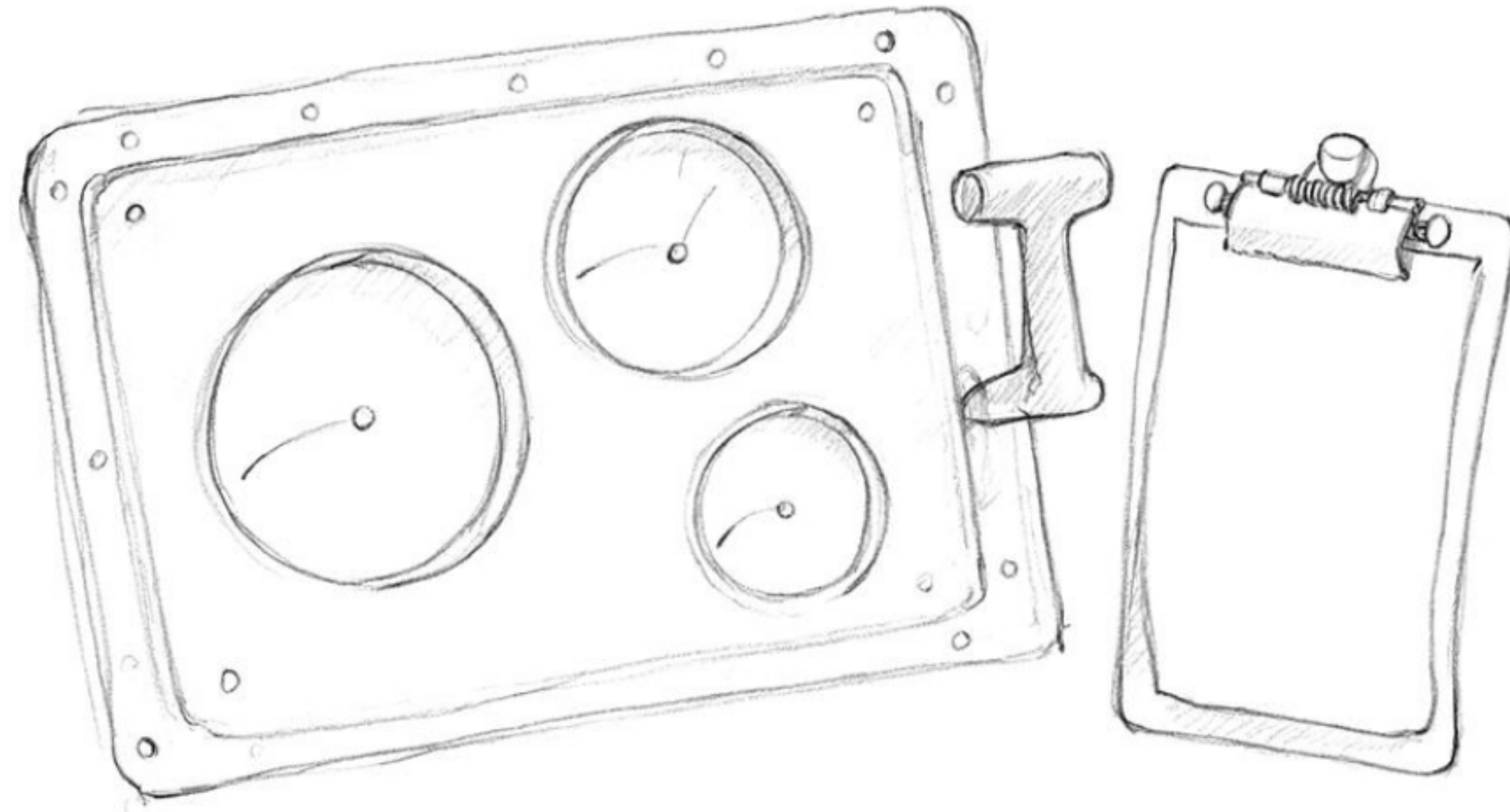
Nothing broke — it worked, and scale costs

The pattern underneath both



not "does this work" — "what does this cost, or how often does it break, at real volume?"

Rare in a test run isn't rare at scale



Evaluation and monitoring: how you know it's actually working

A checklist before every flight, a dashboard while you're in the air

pre-flight checklist

in-flight dashboard

speed altitude fuel

Catch it while it's still small

An eval runs before it ships; monitoring runs while it serves traffic

eval

(Module 2's term)

a fixed set of test cases with
known-good answers, run
before a change ships

monitoring

continuous signals watched
while the system serves
real traffic

Find out before a customer does

Three signals worth watching for a model in production

- 1 latency — how long a prediction takes
- 2 error rate — how often a request fails outright
- 3 prediction drift — outputs trending away from evals
the early warning sign for degradation

The Deep Dive goes further on each one

MODULE 7 TAKEAWAYS

You know the four ways it stops working — today you serve one, for real



- 1 the notebook-to-production gap
- 2 drift, degradation, hallucination at scale, cost
- 3 evals and monitoring
- 4 serving a model behind an API

Gourav Shah · School of DevOps & AI · Now open Module 7 · Lab — Serve Your Model as an
API